COMP3122 Information Systems Development

Group #8

Final Project Report

STEM Learning Platform with GenAI

Prepared by	Group #8
Date	April 2026
Repo snapshot	2026-04-05
Live product	ai-stem-learning-platform-group-8.vercel.app
Demo video	youtu.be/xsq_rRhuclY
Primary repository	HaochenFa/AI-Learning-Platform-for-STEM
Submission repository	comp3122-2526sem2/groupproject-team_8

Group Members

Name	Student ID
FA, Haochen	24113347D
WONG Ka Hei	23118174D
KWAN Kai Man	23118556D
LI, Zhengyang	24099899D

This report documents the current implemented state of the repository, including the web application, Python backend, Supabase schema and Edge Functions, testing strategy, deployment topology, and role-based user workflows.

Contents

1 Executive Summary **3**

2 Project Background and Motivation **3**

2.1 Course and problem context . . . 3

2.2 Project goals . . . 4

2.3 What makes this project distinctive . . . 5

3 System Overview and Implemented Features **5**

3.1 High-level system picture . . . 5

3.2 Primary user groups . . . 6

3.3 Feature set . . . 7

3.4 Blueprint-centered value flow . . . 9

4 System Design **12**

4.1 Technology stack . . . 12

4.2 Architectural decomposition . . . 12

4.3 Web application design . . . 13

4.4 Python backend design . . . 13

4.5 AI orchestration design . . . 14

4.6 Chat workspace and memory design . . . 14

4.7 Material processing pipeline . . . 15

4.8 Analytics and adaptive teaching support . . . 16

4.9 Security and access-control design . . . 17

4.10 Guest sandbox design . . . 17

5 Software Development Process **18**

5.1 Requirements and scope shaping . . . 18

5.2 Design process . . . 18

5.3 Implementation phases observed in git history . . . 19

5.4 Collaboration workflow . . . 19

5.5 Use of GenAI during development . . . 20

6 Testing, Quality Assurance, and Deployment **20**

6.1 Testing strategy . . . 20

6.2 Why the testing approach is appropriate . . . 21

6.3 Deployment topology 21
6.4 Deployment process 22
6.5 Database evolution 23

7 Lessons Learned and Reflection **23**
7.1 Challenges that sharpened the product 23
7.2 Advantages proven during development 24
7.3 Reasonable next-stage directions 25

8 Conclusion **25**

9 User and Operations Manual **27**
9.1 Administrator Manual (Deployment and Operations) 27
9.2 Instructor Manual 30
9.3 Student Manual 32
9.4 Supplementary Guest Demo Flow 34

A Repository Evidence Used for This Report **35**

B Submission and Live System Links **35**

1 Executive Summary

This project delivers a production-oriented STEM learning platform that places Generative AI inside the real classroom operating model rather than exposing it as a detached chatbot. Its core idea is simple but powerful: teachers should be able to transform their own teaching materials into a structured *Course Blueprint*, and that blueprint should govern what students study, what the AI can generate, and what the teacher can review afterwards.

That blueprint-centered loop gives the product its practical value. Teachers upload materials, the system processes and structures them, AI helps generate and refine the class blueprint, students learn through blueprint-grounded chat and activities, and the resulting student work flows back into review, class intelligence, and adaptive teaching support. In other words, AI is used here to support the full teaching cycle from preparation to intervention, not just to answer questions on demand.

The implemented platform serves three concrete audiences. Teachers receive management and decision surfaces for class creation, material curation, blueprint publishing, assignment authoring, submission review, chat monitoring, and teaching insight interpretation. Students receive a separate guided workspace for join-by-code class access, published blueprint study, always-on class chat, and tracked assignment completion. Guests receive a sandboxed but realistic evaluation path with anonymous entry, role switching, reset, and data isolation, making the product demonstrable without touching real classroom data.

Operationally, the system is implemented across three cooperating runtime surfaces: a **Next.js 16** web application for role-aware user experience and server actions, a **Python FastAPI** backend for AI orchestration and workflow-heavy service logic, and a **Supabase** project for authentication, PostgreSQL persistence, row-level security, storage, and background-job support. This split gives the platform clearer operational boundaries and allows AI orchestration, persistence, and lifecycle concerns to be treated as first-class system responsibilities.

The current repository also provides strong evidence of implementation maturity:

- **Live product:** the deployed system is available at `ai-stem-learning-platform-group-8.vercel.app`.
- **Repository scale:** the current snapshot contains 411 commits, including 92 merge commits, alongside 23 active database migrations.
- **Quality signals:** the codebase currently includes 60 web test files, 17 backend test files, 9 Playwright end-to-end specifications, and a documented deployment topology spanning the web app, Python backend, and Supabase services.

Taken together, the current snapshot represents a coordinated educational product with credible implementation depth and clear operational boundaries, not a toy prototype or a one-off AI demo.

2 Project Background and Motivation

2.1 Course and problem context

The course brief frames the project in the broader language of *Education 4.0*, personalized learning, and AI-supported teaching. That framing is useful, but the repository shows that

the real product challenge is more concrete: classroom AI must do more than answer questions. Teachers need a system that can absorb their materials, structure them into trustworthy course knowledge, turn that knowledge into student-facing activities, and still leave the teacher in editorial control.

In practice, the pain points are operational as much as pedagogical. A teacher does not only need "an AI"; they need help with lesson preparation, activity creation, assignment delivery, review, and follow-up instruction. Students, in turn, do not benefit much from a generic chatbot that has no understanding of what their class is actually studying. They need a guided workspace grounded in the same materials, objectives, and assignments that the teacher is using.

This project responds to that problem by treating the class itself as the unit of AI orchestration. Uploaded teaching materials become the basis for a structured Course Blueprint, the blueprint becomes the source of truth for chat and activities, and the resulting student work flows back into review and teacher intelligence surfaces. That is the central product idea that differentiates the system from a thin AI wrapper or a one-off classroom demo.

2.2 Project goals

Based on the implemented system and supporting documentation, the project goals can be expressed as four connected product objectives rather than a flat checklist.

Structured knowledge creation. The first goal is to transform raw classroom materials into reusable, class-specific knowledge. The implemented platform does this through a materials library, asynchronous processing, and AI-assisted blueprint generation so that uploaded files become an organised teaching asset instead of a passive attachment store.

Teacher control and trust. The second goal is to keep instructors in control of what AI produces. The blueprint workflow is therefore not instant-publish generation: teachers edit the draft, approve it into an overview state, and publish it only when it is ready for students. The same idea extends to quiz and flashcard drafts, which are editable before assignment.

Guided student learning. The third goal is to give students a classroom workspace that feels specific to their class rather than generic to the internet. The implemented student experience centres on the published blueprint, always-on class chat, assignable quiz/flashcard/chat activities, and visible progress states such as assigned, in progress, submitted, and reviewed.

Operational credibility. The fourth goal is to demonstrate that the platform is credible as a real software product, not only as a concept. The current repository therefore includes a dedicated backend orchestration layer, queue-backed material processing, deployment documentation, automated tests across web and backend surfaces, and a sandboxed guest evaluation mode that allows realistic product walkthroughs without touching live classroom data.

2.3 What makes this project distinctive

What makes the project distinctive is not any single AI endpoint, but the way several design choices combine into a more complete classroom system.

Blueprint as the pedagogical control layer. The Course Blueprint is not decorative metadata. It is the product's central teaching object: it is generated from class materials, edited by the teacher, versioned through draft/overview/published states, and then reused to ground downstream student experiences. This adds a meaningful editorial checkpoint that most GenAI classroom demos do not have.

A full classroom loop rather than isolated generation. The system supports an end-to-end workflow from teacher setup to student activity to teacher review. That means AI is used not just to generate content, but to support assignment creation, submission handling, feedback, chat monitoring, class insights, and teaching briefs. The product therefore models classroom operations, not only content synthesis.

A deliberate AI orchestration boundary. All AI generation routes through a dedicated Python FastAPI service rather than ad hoc provider calls from the web layer. This gives the system a clearer separation of concerns, consistent response contracts, and a stronger basis for provider routing, validation, and future extension.

Production-minded background and persistence design. Material processing is asynchronous and queue-backed, analytics and teaching briefs are snapshot oriented, and blueprint publication is treated as a state transition rather than a transient prompt result. These choices make the product feel much closer to a deployable platform than to a hackathon prototype.

An authentic guest evaluation path. Guest mode is implemented as a real sandbox classroom with anonymous authentication, seeded demo data, role switching, reset, expiry, and cleanup. That is a significantly stronger demonstration surface than a static landing page tour or a pre-recorded slideshow of product screens.

3 System Overview and Implemented Features

3.1 High-level system picture

Figure 1 shows the top-level structure of the current platform.

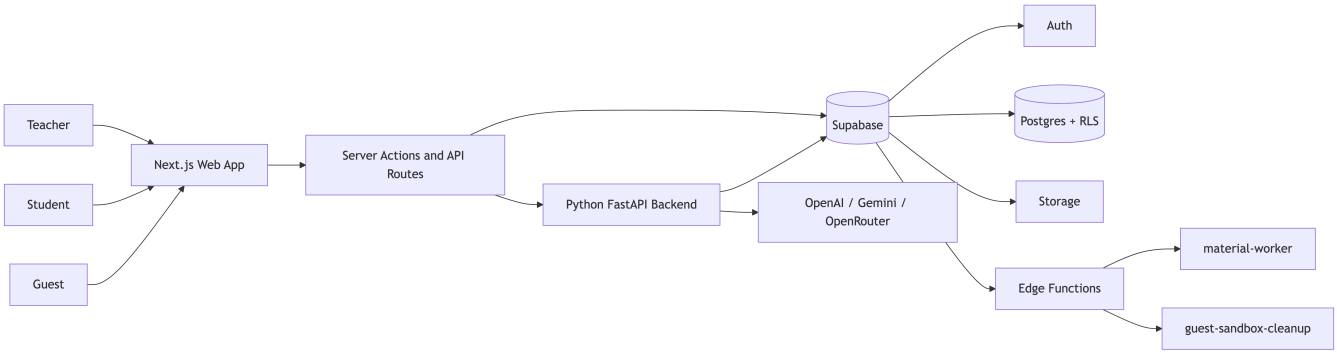

Figure 1: Current high-level platform architecture.

At a high level, teacher, student, and guest users all enter through the web application. The web layer handles UI rendering and user flow orchestration, while the backend is responsible for AI-heavy logic and the Supabase layer provides persistence, storage, security, and async jobs.

3.2 Primary user groups

Table 1: Primary user groups in the current release

Role	Implemented responsibilities
Instructor	Create and manage classes, upload materials, curate and publish blueprints, generate and assign activities, review submissions, monitor chat behaviour, inspect insights, use the adaptive teaching brief, and preview the student journey from within the class workspace.
Student	Join classes with a join code, study the published blueprint, use always-on class chat, complete assigned chat/quiz/flashcard activities, and track current, upcoming, completed, and reviewed work from the student dashboard.
Guest	Enter a sandbox classroom through anonymous auth, explore seeded teacher and student workflows through role switching, reset the demo when needed, and evaluate the product without affecting real user data.

These user groups are not theoretical labels; they map directly to different product surfaces and navigation models in the implementation.

Instructor. The instructor experience is creation-first and decision-heavy. Teachers land in a dedicated dashboard, see their owned and assistant classes, and move into a class shell where they can upload materials, generate blueprints, create assignments, monitor student chats, and interpret class insights. The product also supports preview-as-student mode, which lets a teacher inspect the student-facing experience without leaving the classroom context.

Student. The student experience is intentionally simpler and more guided. Students enter through a separate dashboard, join classes through join codes, and then work inside a class workspace organised around the published blueprint, always-on chat, and assigned activities.

Their dashboard and class pages emphasise due work, progress state, and reviewed feedback rather than creation or analytics tools.

Guest. The guest experience is designed for realistic evaluation rather than passive observation. Guests authenticate anonymously, receive a seeded sandbox classroom, and can switch between teacher and student perspectives, reset the environment, or convert to a real signup path. Middleware and sandbox state keep the guest confined to the sandbox classroom so the demo remains safe and operationally believable.

In addition to these primary personas, the implementation also supports a class-scoped **teaching assistant (TA)** authorization role. TA is not exposed as a separate permanent signup account type; instead, it exists as an enrollment-level role that can access the same teacher-oriented class workflows where route and service logic explicitly permit `teacher` or `ta`.

3.3 Feature set

Rather than presenting the system as a single AI chatbot, the current release delivers a set of user-facing classroom capabilities that work together as a guided teaching and learning platform. The most important design decision is that AI is always embedded inside a teacher-controlled class workflow: content is grounded in uploaded materials, reviewed through a blueprint, released through activities, and then interpreted through feedback and insights. The main implemented feature groups are summarised below.

Role-Based Access and Entry

The platform supports distinct entry paths for teachers and students rather than a single generic login followed by ad hoc navigation. Permanent users sign in with email and password, complete email verification, and then land in role-specific dashboards that match their responsibilities. Teachers enter management-oriented flows for classes, materials, blueprints, assignments, and insights; students enter a simpler workspace centred on class participation and assigned work.

This access model is implemented as a shared authentication surface rather than duplicated forms. The same auth component handles sign-in, sign-up, and password recovery in both the landing-page modal and dedicated auth pages, while server-side confirmation and recovery routing supports secure email verification and password reset completion. The result is a cleaner product experience for users and a more maintainable authentication boundary for the system.

Classroom Setup and Participation

For teachers, the first meaningful product workflow is class setup. A teacher can create a class, define its basic metadata, and immediately receive a join code that can be shared with students. Students can then join an existing class through a join-by-code flow and are placed directly into a class-scoped workspace with the right navigation, permissions, and learning surfaces for their role.

This capability is more than a simple CRUD screen. Class creation automatically enrolls the creating teacher, join codes are validated through a dedicated backend workflow, and join operations are designed to be safely repeatable rather than fragile one-time mutations. Once

inside a class, the platform keeps users in a consistent class shell, and teachers can even preview the student experience in read-only mode without leaving the same classroom context.

Material Ingestion and Blueprint Publishing

The core differentiator of the platform is its material-to-blueprint workflow. Teachers can upload PDF, DOCX, and PPTX teaching materials into a class library, monitor their processing status, and manage them through preview, download, and delete actions. This turns the class workspace into a real content hub rather than a thin wrapper around a text prompt box.

Once materials are available, the platform uses them to generate a structured Course Blueprint that becomes the class source of truth. The generated blueprint is not exposed to students immediately: teachers edit the draft, adjust topics, objectives, and prerequisite structure, approve it into an overview state, and then publish it when ready. This creates a clear editorial checkpoint and makes the AI output reviewable rather than automatic.

The implementation depth behind this feature is substantial. Material ingestion is asynchronous, with extraction, chunking, and embedding handled in the background so the system does not pretend that an uploaded file is instantly AI-ready. Blueprint generation produces structured payloads rather than free-form prose, while publication is owner-controlled and version-aware, allowing a teacher to create a new draft from the currently published blueprint without losing the stable student-facing version until the revised content is explicitly promoted.

AI Learning Activities for Students

After a blueprint is published, the platform turns that curated course knowledge into multiple learning experiences for students. Students can use an always-on class chat for open practice, and teachers can create formal chat, quiz, and flashcard assignments from the class blueprint. This gives the product a broader learning model than a one-dimensional question-answer interface.

Quiz and flashcard activities are generated as editable drafts, allowing teachers to refine the AI output before publishing it and assigning it to the whole class. Chat assignments add teacher-authored instructions to a guided conversation flow, while student dashboards and class views track assigned, in-progress, submitted, and reviewed states. Quiz flows support repeated attempts and score tracking, flashcard sessions capture known-versus-needs-review outcomes, and chat assignments preserve both the conversation transcript and student reflection for later review.

The student experience is also visually richer than plain text output. In both chat and data-rich contexts, the system can attach generative canvas responses such as charts and diagrams when a concept is better explained visually. This is especially valuable in STEM learning, where ideas such as processes, relationships, and trends are often easier to understand when rendered instead of only described.

Teacher Review and Instructional Intelligence

The platform does not stop at content generation or assignment delivery. Teachers can open dedicated review screens for chat, quiz, and flashcard assignments to inspect what students

actually submitted, check completion status across the class, and record feedback. This shifts the product from “AI makes content” to “AI supports a full teaching cycle”.

The review experience is implemented with meaningful depth. Teachers can inspect quiz attempts and answers, read chat transcripts, review flashcard session outcomes, override or adjust scores where appropriate, and save comments plus highlights back to the student record. In parallel, a read-only chat monitor lets teachers inspect students’ always-on class chats for coaching and support without interfering with the student’s active workspace.

On top of review, the platform provides a teacher intelligence layer. The class intelligence dashboard aggregates quiz performance, topic outcomes, Bloom-level coverage, engagement patterns, student drill-downs, and intervention suggestions. A natural-language data-query panel lets a teacher ask questions about class performance and receive generated charts, while the adaptive teaching brief condenses the current class state into a memo-style summary with attention items, students to watch, likely misconceptions, strongest recommended action, and suggested next steps. These features are backed by persisted snapshots so teacher-facing summaries remain stable across reloads and can be refreshed intentionally instead of regenerating unpredictably on every visit.

Guest Sandbox Evaluation Mode

Guest mode is one of the most distinctive parts of the release because it is implemented as a real interactive sandbox rather than a fake product tour. A visitor can enter from the landing page, authenticate anonymously, and receive a cloned classroom seeded with demo materials, blueprint data, activities, analytics, and chat history. This allows stakeholders to experience realistic teacher and student flows without touching live user data.

The sandbox experience includes role switching between teacher and student perspectives, reset capability for restarting the demo, and clean handoff into permanent signup when a guest wants to become a real user. It is also operationally disciplined: guest sessions are isolated through sandbox-scoped data, confined to allowed routes, limited by global session quotas, and expired after inactivity or maximum lifetime. A dedicated cleanup workflow then removes guest data, associated storage objects, and anonymous auth users so the feature remains sustainable in a hosted environment rather than becoming a one-off demo hack.

3.4 Blueprint-centered value flow

The most important end-to-end educational flow in the current release is shown in Figure 2. Unlike the lighter overview used in the repository README, this workflow captures the implemented control points that make the platform educationally credible: background material readiness, teacher-governed blueprint publication, blueprint-gated activity delivery, persisted student work, and teacher-facing instructional response.

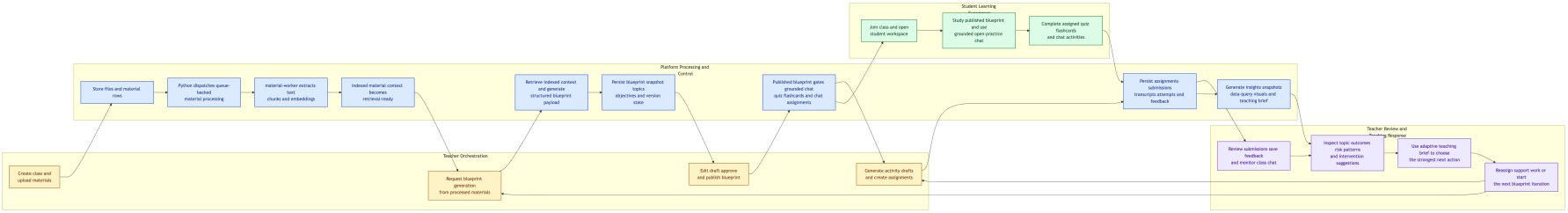

Figure 2: Implemented blueprint-governed classroom workflow from material ingestion to student activity, review, and teaching intervention.

Material ingestion and readiness. The flow begins with teacher-owned classroom content rather than with a prompt box. When an instructor uploads PDF, DOCX, or PPTX materials, the web application stores the file and material record immediately, but it does not pretend the content is AI-ready at once. Instead, the Next.js server action dispatches a background processing request to the Python backend, which enqueues the work and can trigger the Supabase `material-worker` Edge Function. That worker extracts text, chunks it, generates embeddings, and persists retrieval-ready material context. This is an important implementation choice because it turns uploaded files into operational class knowledge through a real asynchronous pipeline, with clear `processing`, `ready`, and `failed` states, rather than through a fragile synchronous shortcut.

Blueprint generation and publication control. Once materials are ready, the platform retrieves indexed class context and sends it to the Python backend for structured blueprint generation. The backend does not return informal prose; it returns a validated payload containing summary, topics, objectives, prerequisite structure, and related teaching metadata. The web layer then persists that output both as a canonical blueprint snapshot and as normalized relational records for topics and objectives. Crucially, this generated blueprint is not student-facing by default. Teachers edit the draft, validate topic relationships, approve it into an overview state, and publish it only when they are satisfied with the course structure. The owner controls approval and publication, and the implementation also supports creating a new draft from the published version so the live student-facing blueprint remains stable until a revised version is explicitly promoted.

Blueprint-governed learning delivery. After publication, the blueprint becomes the governing instructional artifact for downstream AI features. Open-practice class chat, quiz generation, flashcard generation, and teacher-created chat assignments all depend on the published blueprint rather than bypassing it. This is where the product's core idea becomes concrete: students do not interact with a generic model that improvises outside the class context; they interact with AI features constrained by approved course knowledge and retrieved material evidence. The implementation distinguishes between activity types in a sensible way. Quiz and flashcard activities are generated as editable drafts so teachers can refine the AI output before release, while chat assignments publish directly because the teacher is already supplying the assignment framing and instructions in one step.

Student work and teacher review. The workflow then moves from content delivery into accountable classroom participation. When a teacher assigns an activity, the system creates assignment records for enrolled students and tracks their status across states such as assigned, in progress, submitted, and reviewed. Students study the published blueprint, use always-on grounded class chat, and complete quiz, flashcard, and chat assignments inside a class-scoped workspace. Their attempts, transcripts, reflections, and feedback artifacts are persisted rather than discarded after generation. This persistence matters because it allows teachers to inspect actual student work through review screens, save comments and highlights, check quiz attempts and outcomes, and monitor always-on class chat activity without interrupting the student's learning flow.

Insights and adaptive teaching response. The loop closes when student activity becomes instructional intelligence rather than raw interaction data. Persisted submissions and chat outcomes feed analytics snapshots, visual data-query responses, risk indicators, topic-level performance summaries, and intervention suggestions. On top of that, the adaptive

teaching brief condenses current class evidence into a memo-style teaching response with attention items, misconceptions, students to watch, strongest recommended action, and suggested next steps. This means the platform is not merely generating classroom content; it is helping the teacher interpret what happened and decide what to do next.

Taken together, this flow is the clearest expression of the product's value. The platform does not stop at AI-assisted content creation, and it does not treat classroom interaction as disposable chat traffic. Instead, it implements a governed instructional cycle in which teacher materials become structured class knowledge, that knowledge governs student activity, student activity produces reviewable evidence, and that evidence informs the teacher's next instructional move.

4 System Design

4.1 Technology stack

Table 2: Core implementation stack

Layer	Technology
Frontend	Next.js 16 App Router, React 19, TypeScript, Tailwind CSS 4, shared UI primitives, Motion for React, Lucide icons
Backend	Python FastAPI, httpx, structured Pydantic schemas, provider adapters
Database and auth	Supabase PostgreSQL, Supabase Auth, Row Level Security
Storage and async processing	Supabase Storage, pgmq queue, pg_cron, Supabase Edge Functions
Testing	Vitest, React Testing Library, Python unittest, Playwright
Deployment model	Separate web and backend deployments plus hosted Supabase

4.2 Architectural decomposition

The platform is deliberately partitioned into four top-level areas that map cleanly to runtime responsibility:

- `web/`: user-facing routes, UI components, middleware, and server actions;
- `backend/`: AI orchestration, chat workspace logic, class workflows, analytics;
- `supabase/`: SQL migrations, local config, and Edge Functions;
- `tests/`: Playwright end-to-end coverage running against a deployed target URL.

This separation is important because each layer owns the work it is best positioned to execute. The web application owns rendering, navigation, and server-action orchestration. The Python backend owns AI routing, chat-workspace behavior, analytics, and workflow-heavy service logic. Supabase owns authentication, PostgreSQL persistence, row-level security, storage, and

queue-backed background operations. The result is a clearer product boundary between interface concerns, orchestration concerns, and stateful platform concerns.

4.3 Web application design

The web application delivers the platform's role-aware operating surface. It separates teacher and student experience from the first navigation step, keeps users inside a persistent class shell once they enter a classroom, and uses server actions plus middleware to make state transitions feel consistent across the product.

Important web-layer characteristics include:

- role-specific dashboards at `/teacher/dashboard` and `/student/dashboard`;
- a home-first auth model via a shared `AuthSurface` used in the landing-page modal and dedicated routes such as `/login`, `/register`, and `/forgot-password`;
- middleware-enforced redirects for unauthenticated, unverified, and expired guest sessions;
- server actions for class creation, join-by-code, materials, assignments, and settings;
- a shared design system with centralized icons, tokens, and motion presets.

On the teacher side, the class overview acts as the operational control hub for materials, assignments, teaching brief, and class monitoring. On the student side, the class workspace is organized around focused learning surfaces for chat, assignments, and the published blueprint rather than teacher-oriented management tools. A teacher can also preview the student experience from within the same classroom context, which is a strong product-control detail because it lets an instructor inspect the student-facing flow without changing accounts or leaving the class shell.

The authentication surface is also more disciplined than a standalone login form. The platform uses the same shared auth presenter for landing-page modal entry and fallback dedicated routes, supports server-side email confirmation and password-recovery callbacks through `/auth/confirm`, and completes recovery through a dedicated `/reset-password` flow. Reusing the same auth component and redirect model across public entry points and email-link return paths reduces UI drift and keeps authentication behavior easier to maintain.

4.4 Python backend design

The Python backend serves as the platform's AI and workflow control plane. That is a significant architectural choice because it keeps prompt construction, provider routing, response normalization, and service-to-service validation out of the web layer and behind a dedicated FastAPI boundary.

The backend provides endpoints and domain logic for:

- blueprint generation;
- quiz generation;
- flashcard generation;
- grounded chat;

- chat workspace sessions and message persistence;
- visual canvas generation;
- class create and class join;
- materials dispatch;
- analytics, teaching brief, and natural-language data queries.

All normal backend responses follow a consistent envelope:

```
{ ok, data, error, meta }
```

That contract gives the web adapters a uniform integration surface for success, failure, and request metadata, while allowing the backend to evolve internal provider logic, prompt structure, and workflow composition without forcing parallel changes across many frontend call sites.

4.5 AI orchestration design

AI orchestration is environment-driven and intentionally centralized in the backend. The platform supports three pluggable providers — OpenRouter, OpenAI, and Gemini — and the provider layer resolves an ordered attempt list from the configured default plus the providers that are actually usable in the current environment.

This design has several advantages:

- the web app remains provider-agnostic;
- deployment environments can switch provider preference through configuration;
- provider failures can be isolated inside the backend;
- request deadlines are enforced centrally rather than inconsistently per route.

The orchestration layer is also more robust than a simple "try another API key" fallback. It applies a single wall-clock deadline across provider attempts, recomputes the remaining budget before each call, and uses the same ordered fallback approach for both generation and embeddings. Operationally, that means a deployment can prefer one provider by configuration, degrade to another when necessary, and still preserve a stable frontend contract.

The AI surface is broader than content generation alone. In addition to blueprint, quiz, and flashcard generation, the backend also supports grounded chat, analytics narratives, teaching brief synthesis, embeddings, and visual canvas specifications. This gives the platform a unified AI layer for both student learning workflows and teacher decision-support workflows.

4.6 Chat workspace and memory design

The chat workspace is one of the platform's most technically mature subsystems because it is designed as a long-lived classroom conversation system rather than a disposable prompt-response loop. The backend supports participant listing for teacher monitoring, session

creation, rename and archive operations, paginated message history, and persisted assistant responses so a class conversation can continue over time instead of resetting on every visit.

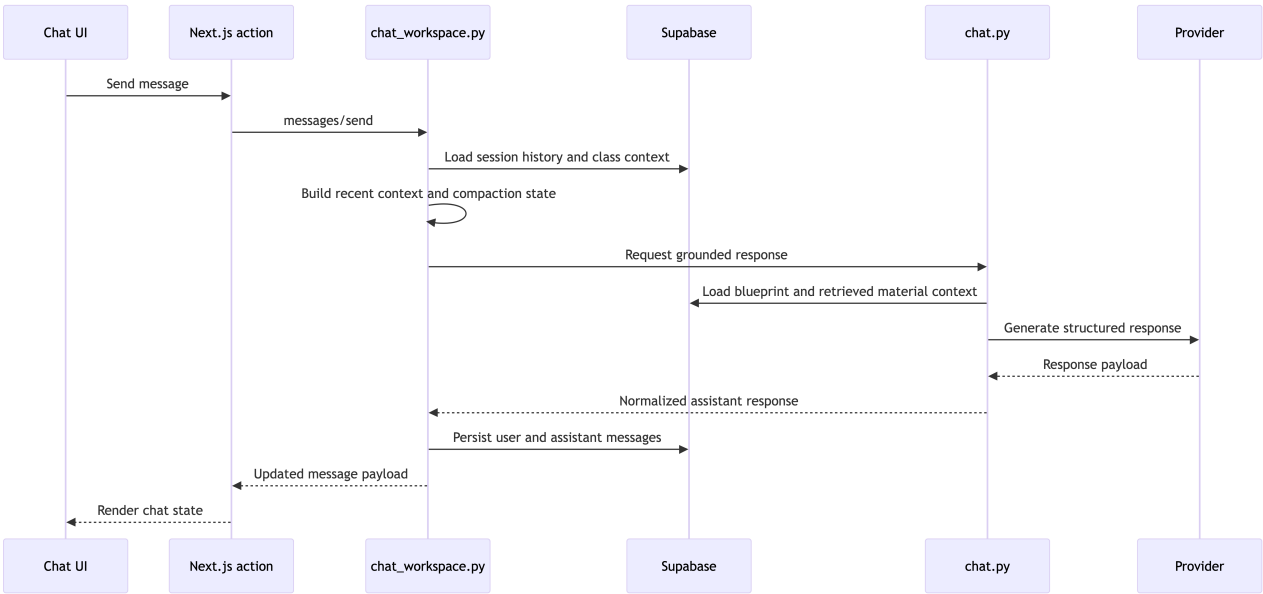

Figure 3: Chat workspace flow with grounded context retrieval and persisted assistant responses.

The grounding path is equally important. Workspace chat is constrained by published blueprint context and retrieved material chunks rather than a generic free-form prompt, which keeps class chat tied to approved course knowledge. For longer conversations, the system uses summary-based compaction and context-window management instead of replaying the full transcript indefinitely. That combination of persistence, grounding, and compaction makes the chat surface much closer to a real classroom tool than to a one-shot chatbot widget.

4.7 Material processing pipeline

Material ingestion is implemented as a real asynchronous processing pipeline. Uploading a file creates the material record and stores the source asset immediately, but AI-readiness is produced through a separate dispatch-and-worker path rather than by blocking the teacher's request until extraction and embedding complete.

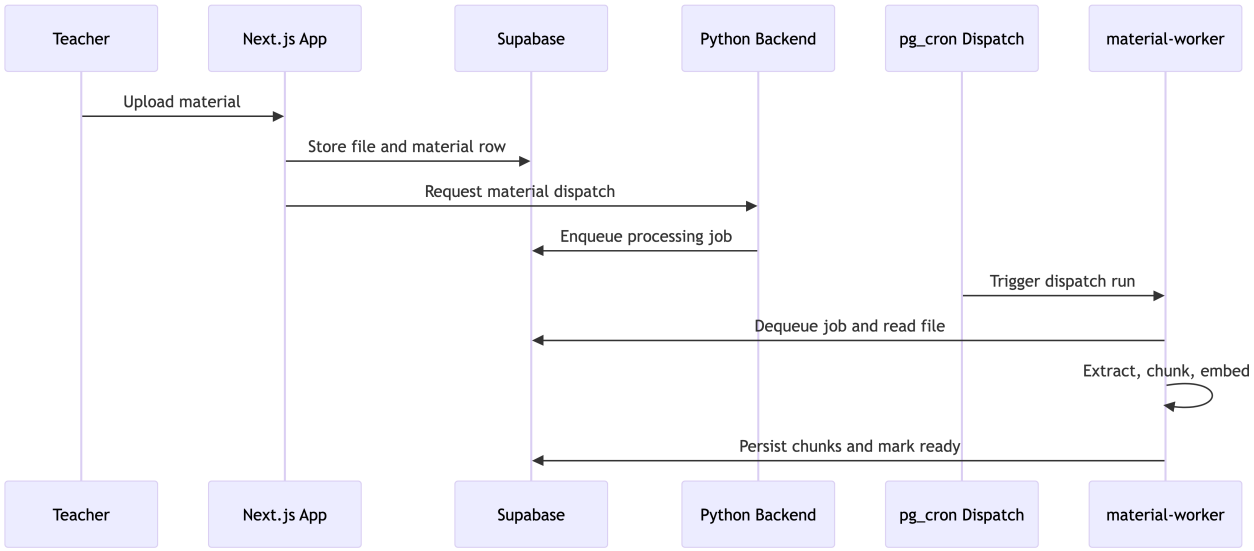

Figure 4: Asynchronous material-processing pipeline.

The implemented flow is more substantial than a background task checkbox. The web layer persists the material, the Python backend calls an `enqueue_material_job` RPC, PostgreSQL writes the job into a `pgmq`-backed queue, and the Supabase `material-worker` Edge Function extracts text, chunks content, generates embeddings, and writes retrieval-ready records plus status updates. The worker can be awakened immediately after dispatch or picked up through cron-backed execution, which gives the system both responsiveness and resilience.

This design matters because material processing is exactly the kind of workload that becomes fragile when treated as a synchronous request. Queue-backed execution improves retry behavior, makes `processing`, `ready`, and `failed` states explicit in the product, and turns raw classroom files into indexed instructional context rather than opaque attachments.

4.8 Analytics and adaptive teaching support

Teacher-facing intelligence is a meaningful part of the platform design, not a thin reporting add-on. The analytics layer turns persisted submissions, activity outcomes, and chat-derived signals into a decision-support surface for instructors.

The class intelligence dashboard includes:

- summary cards for student count, average score, completion rate, and risk indicators;
- topic-level performance charts;
- Bloom-level breakdown visualization;
- student engagement views and student overview tables;
- AI-generated narrative interpretation and intervention suggestions;
- natural-language data query support for generating chart specifications.

These features are backed by persisted insight snapshots and caching logic so the teacher is not looking at a completely re-generated story on every reload. The adaptive teaching brief

complements the larger dashboard by delivering a memo-style summary directly on the class overview page, with recommended next steps, likely misconceptions, students to watch, and follow-up activity guidance. Together, the dashboard and brief show that the product is designed to help teachers interpret class evidence, not only generate student content.

4.9 Security and access-control design

Security is layered across all three runtime surfaces rather than delegated to any single framework:

- **web layer**: route protection, email verification enforcement, and guest route confinement;
- **backend layer**: service authentication, user bearer-token verification, and guest AI guardrails;
- **Supabase layer**: row-level security, teacher/enrollment policies, sandbox-aware isolation, and cleanup-capable SQL services.

Several implementation details show that this hardening model is intentional rather than cosmetic:

- permanent users are limited to teacher or student account types;
- `profiles.account_type` is immutable after signup;
- class-scoped teacher permissions can also be granted through the enrollment-level `ta` role without turning TA into a separate permanent account type;
- guest sessions are scoped by `sandbox_id`;
- cursor inputs to PostgREST are validated before interpolation;
- service-to-service auth between web and backend uses a shared backend API key.

The combined effect is important. Route guards shape who can enter a workflow, backend verification checks who is calling sensitive service endpoints, and RLS plus sandbox-aware policies determine what data each actor can actually read or mutate. This is the right design for a platform that mixes real classroom data, AI workflows, and anonymous guest sandboxes in the same hosted system.

4.10 Guest sandbox design

Guest mode is one of the product's strongest differentiators because it is implemented as a real interactive sandbox rather than as a scripted tour. The platform creates an anonymous Supabase auth session, provisions a `guest_sandboxes` record, and clones seeded classroom content into normal application tables under a sandbox-specific `sandbox_id`. Guests can then switch between teacher and student views, reset the demo, and experience realistic product behavior without touching real user data.

The quota and lifecycle model combines Supabase-side and backend-side guardrails:

- a Supabase-managed `guest_session_quota` control table with RPCs that gate sandbox creation and quota release;
- default admission caps of 60 active guest sandboxes globally and 20 newly created sandboxes per hour;
- backend per-feature guest AI limits plus an `asyncio.Semaphore`-based concurrency gate, with a default ceiling of 20 in-flight guest AI requests;
- expiry after 32 hours maximum or 8 hours of inactivity.

Cleanup is part of the design, not an afterthought. The `guest-sandbox-cleanup` Edge Function and related SQL cleanup paths release quota counters, delete guest storage objects and sandbox-scoped class data, and remove anonymous auth users so that demo sessions do not leak storage, quota state, or stale identities over time.

This design gives the platform a much stronger demonstration surface than a read-only walkthrough would provide, while still keeping safety, resource control, and lifecycle hygiene under explicit system control.

5 Software Development Process

5.1 Requirements and scope shaping

The initial project brief pushed the team toward a stakeholder-aware educational system rather than toward a shallow AI showcase. The resulting product scope is therefore not a universal learning platform or a generic study chatbot. It is a structured STEM classroom assistant built around teacher materials, teacher editorial control, and student-facing learning workflows grounded in the same class knowledge.

A practical strength of the delivery was scope discipline. The team did not try to implement every possible education feature. Instead, it converged on one coherent classroom operating loop:

- content ingestion;
- blueprint generation and publishing;
- student chat and assignments;
- review and analytics;
- guest evaluation flow.

That restraint is important because it concentrated engineering effort on one integrated product system instead of spreading effort across unrelated AI experiments.

5.2 Design process

The design process was iterative, documentation-backed, and architecture-aware. The project maintained:

- top-level design, architecture, UI/UX, and deployment documents;

- internal reference notes under `.codex/`;
- Mermaid diagrams exported to PNG for communication and reporting;
- commit history showing multiple phases of refinement rather than one-shot implementation.

UI and UX were also treated as a first-class engineering workstream rather than as late cosmetic polish. The development record shows repeated investment in design tokens, navigation restructuring, shared auth-surface redesign, teacher-versus-student flow separation, and consistency hardening. That matters in an educational product because quality depends not only on AI correctness, but also on clarity, trust, and ease of use in everyday classroom operation.

5.3 Implementation phases observed in git history

The git history shows staged delivery rather than a single end-loaded implementation push. Table 3 summarizes the major development phases visible across the repository.

Table 3: Major development phases observed from git history

Period	Main progression visible in the repository
2026-02-03 to 2026-02-05	Repository foundation, baseline schema and RLS, initial class flows, materials ingestion, provider adapters, blueprint generation, and first UI scaffolding.
2026-02-06 to 2026-02-10	End-to-end vertical slices for grounded chat, quiz generation, flashcards, auth hardening, and always-on class chat workspace.
2026-02-22 to 2026-03-02	Material-worker stabilization, queue-backed processing, live material readiness feedback, class-shell UX improvements, design-token refactoring, and stronger web test coverage.
2026-03-10 to 2026-03-15	Python-backend migration and hardening, canonical backend envelope enforcement, service-auth tightening, materials file management, class intelligence delivery, and architecture/documentation alignment.
2026-03-18 to 2026-03-29	Generative canvas, adaptive teaching brief, review-surface improvements, guest mode rollout, E2E expansion, shared auth redesign, branded email flows, and OTP-based password change.
2026-03-30 to 2026-04-05	Inline code documentation, analytics fixes, guest quota redesign, concurrency hardening, environment/document refresh, diagram alignment, and final report refinement.

The overall delivery pattern is clear: establish the domain model, build the main classroom slices, consolidate the backend and runtime boundaries, expand teacher-support and guest-demo capabilities, and then harden the system, documentation, and submission artifacts near the end of the project.

5.4 Collaboration workflow

The collaboration model combined clear primary technical leadership with meaningful supporting contributions across the rest of Group #8. Haochen Fa served as the primary

engineer and solution architect, leading the overall technical direction, system architecture, cross-service integration, deployment alignment, and most feature delivery. Other team members contributed through code review, feature validation, edge-case testing, product feedback, documentation work, and selected implementation tasks.

GitHub was the formal engineering workspace for the project. The team used GitHub Issues and pull requests to track implementation work, review changes, and communicate technical decisions, while WhatsApp was used for broader day-to-day discussion, rapid coordination, and design feedback. The main development repository is `HaochenFa/AI-Learning-Platform-for-STEM`, and the organization repository prepared for review and submission is `comp3122-2526sem2/groupproject-team_8`.

The engineering record reinforces this workflow. The project history now spans more than 400 commits, including 92 merge commits, nine tagged release checkpoints from `v1.0.0` through `v1.4.0`, and repeated PR-based integrations through at least PR #102. Those merges cover major workstreams such as guest mode, auth redesign, UI/UX improvements, analytics, documentation refreshes, email flows, and final runtime hardening. This is the profile of a staged collaborative build with regular integration checkpoints, not a single large delivery at the end.

5.5 Use of GenAI during development

GenAI-assisted development was used deliberately as part of the delivery strategy rather than as a substitute for engineering ownership. Codex and Claude Code were used to accelerate scaffolding, refactoring, debugging, documentation maintenance, test-support work, and repetitive implementation tasks across the web, backend, and Supabase surfaces. The pull-request workflow also incorporated GitHub Copilot review as an additional automated review layer.

These tools were effective because they operated under disciplined human direction. Clear constraints, architectural context, repo-specific instructions, and fast technical validation turned them into productive accelerators. Without that experienced human harness, the same tools would not have produced work at the same level of alignment, efficiency, or reliability.

Generated suggestions were therefore always paired with human review. Outputs were checked against feature behavior, code structure, testing obligations, Git and GitHub workflow hygiene, dependency-tooling correctness, and the runtime requirements of the Next.js web app, the FastAPI backend, and the Supabase layer. In that sense, prompt engineering and AI-tool orchestration became part of the team's engineering process, while responsibility for correctness, maintainability, and product quality remained fully human.

6 Testing, Quality Assurance, and Deployment

6.1 Testing strategy

The platform uses a layered testing strategy rather than relying on a single proof mechanism.

Table 4: Current testing layers in the repository

Layer	What it covers	Evidence in repo
Web unit and integration tests	Server actions, auth flows, route guards, material actions, blueprint logic, chat flows, guest helpers, UI primitives, and validation helpers	60 test files under `web/src`
Backend unit tests	Providers, schemas, class flows, blueprint generation, quiz, flashcards, materials, analytics, chat workspace, and guest rate limiting	17 Python test files under `backend/tests`
End-to-end tests	Teacher navigation, student navigation, login, join class, settings, sign-out, and guest entry against a deployed environment	9 Playwright specs under `tests/e2e`

Representative quality signals include:

- guest middleware tests plus guest sandbox and quota tests;
- blueprint action tests;
- materials extraction, preview, retrieval, and action tests;
- chat generation, compaction, and workspace tests;
- analytics, insights, and teaching brief action tests;
- Playwright flows across teacher, student, settings, auth sign-out, and guest entry.

6.2 Why the testing approach is appropriate

This test distribution matches the system's runtime boundaries:

- unit tests catch domain logic regressions close to the code;
- web integration tests validate server actions and route behavior;
- backend tests verify AI-related orchestration and guardrails without needing full browser runs;
- Playwright tests confirm that the deployed user flows still work end to end.

For a system that spans a Next.js application, a separate FastAPI service, hosted Supabase state, and queue-backed workers, this layered model is the right testing shape. It contains regressions at the domain layer, verifies critical server-side workflow behavior, and still proves that the hosted teacher, student, and guest journeys function correctly after deployment.

6.3 Deployment topology

The hosted platform is deployed as three coordinated surfaces: a frontend deployment, a separate Python backend deployment, and a hosted Supabase project. Figure 5 captures this structure.

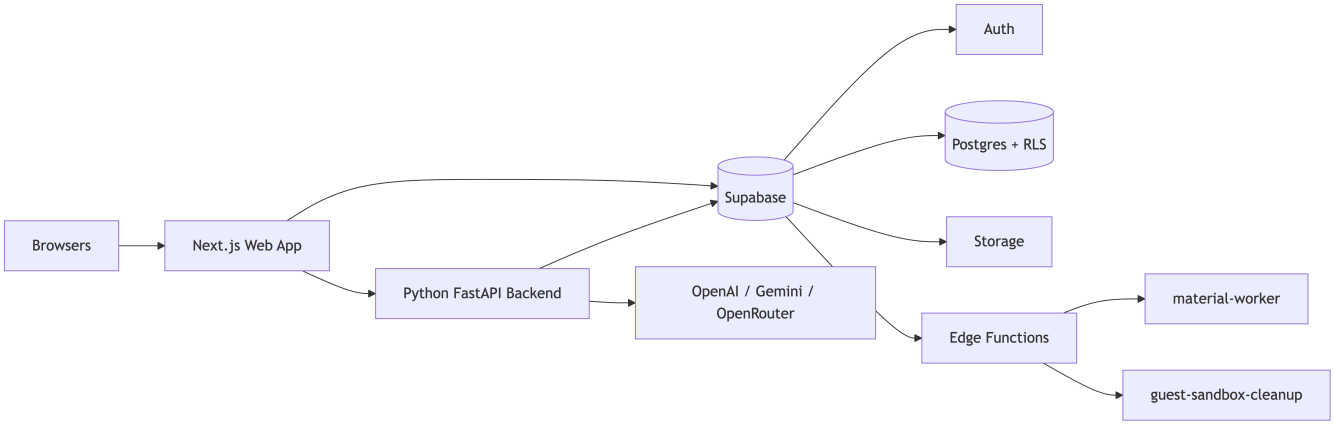

Figure 5: Hosted deployment topology across frontend, backend, and Supabase services.

This topology gives the product clearer operational boundaries:

- frontend behavior can be separated from backend or data-plane faults during debugging;
- AI orchestration can evolve independently from the web deployment surface;
- Supabase-specific responsibilities such as migrations, Edge Functions, and secrets remain explicit;
- preview and staging environments can be aligned more carefully across the stack.

The deployment runbook also makes clear that this is not a nominal split. Both `web/` and `backend/` carry Vercel project linkage, while Supabase hosts authentication, PostgreSQL, storage, RLS policies, SQL dispatch helpers, and the `material-worker` and `guest-sandbox-cleanup` Edge Functions. In other words, the deployed system is genuinely multi-surface, not a frontend-plus-database simplification.

6.4 Deployment process

The documented deployment workflow includes:

1. configuring one Supabase project per environment;
2. applying the active migration set;
3. deploying the `material-worker` and `guest-sandbox-cleanup` Edge Functions;
4. configuring Edge Function secrets plus the Vault-backed values used by SQL-side dispatch;
5. deploying the Python backend;
6. deploying the web application;
7. running post-deploy smoke tests for auth, teacher flows, student flows, insights, and guest mode.

Operationally, this matters because deployment is not just a frontend publish. The schema, data plane, background workers, backend environment contract, and post-deploy smoke checks all have to line up before the product is truly usable. The material pipeline, in particular, depends on queue dispatch, worker execution, embedding configuration, and secret placement across multiple surfaces.

Guest mode makes that need for alignment even more visible. The hosted environment has to keep the Supabase quota RPCs, guest schema, backend guest-limit environment variables, anonymous-auth configuration, and cleanup worker synchronized so that session admission, inactivity expiry, and AI-concurrency guardrails behave consistently across the stack.

6.5 Database evolution

The active schema history now contains 23 migrations, which documents iterative platform evolution rather than a static schema frozen after the first implementation pass.

Major migration themes include:

- initial schema and RLS foundation;
- quiz integrity and auth hardening;
- canonical blueprint snapshots and publication hardening;
- always-on class chat and context compaction;
- queue-backed material processing;
- class insights and teaching brief snapshots;
- guest mode schema, seed data, cleanup, and quota redesign.

This migration history matters because it records the product's growth in a traceable and reviewable form. The database changes do not read like generic table churn; they align closely with major capability additions such as blueprint governance, persistent chat, async material processing, teacher intelligence, and sandboxed guest evaluation.

7 Lessons Learned and Reflection

7.1 Challenges that sharpened the product

The most useful lesson from the development record is that the hardest parts of the platform were not always the most visible ones. On the surface, the system looks like a clean teacher-student learning product with AI assistance. In the commit history, however, the real engineering effort is visible in the repeated cycles of correction, hardening, and redesign that turned that product idea into something dependable. The guest-mode work is the clearest example. What appears in the UI as a simple "continue as guest" action ultimately required anonymous authentication, sandbox cloning, quota governance, expiry handling, cleanup flows, backend ownership checks, and later a redesign from per-IP entry limiting to a global session-quota model. That progression is important because it shows that the team did not stop at a demo-friendly implementation; it kept refining the subsystem until it behaved like a controlled production feature.

Authentication and account recovery taught a similar lesson about trust. The shared auth surface now looks cohesive, but the repository history shows that getting there required several iterations: home-first auth redesign, resend and confirmation recovery flows, guest-signup loop fixes, branded email templates, and OTP-based password reauthentication for sensitive settings changes. The lesson here is that user trust is built through a chain of small reliability decisions, not through one large feature. In an education platform, where teachers and students depend on predictable access, these flows are part of the core product, not peripheral polish.

Another lesson was that multi-surface deployment demands operational discipline very early. Once the platform was split across the Next.js web app, the FastAPI backend, Supabase storage and database, and two Edge Functions, configuration mistakes stopped being minor inconveniences and became real delivery risks. The repository's maintained notes about `trust_env=False`, Edge Function secret placement, queue-worker configuration, and environment alignment are not incidental trivia. They record issues that only become visible when a system is actually being deployed and debugged across several runtime boundaries. The team effectively learned that architecture is only as strong as its operational contract.

Finally, the project reinforced that AI features become dependable only when they are normalized into product workflows. Blueprint generation, analytics narratives, teaching briefs, and chat responses could not remain raw model outputs. They had to be validated, cached, persisted, versioned, or constrained by class context before they were trustworthy enough for real use. The analytics fixes, teaching-brief cleanup, chat compaction logic, and blueprint publication controls all point to the same lesson: GenAI becomes valuable in a learning system only after software engineering disciplines it into repeatable application behavior.

7.2 Advantages proven during development

The strongest advantage proven during development is the product's coherence. The repository did not drift into a loose collection of unrelated AI experiments because the Course Blueprint stayed at the center of the design. Materials ingestion, blueprint editing, activity generation, student chat, review, and teacher insight surfaces all continue to connect back to the same instructional object. That coherence is visible not only in the current codebase but also in the sequence of feature delivery: new capabilities were repeatedly added by extending the blueprint-centered learning loop rather than by bypassing it.

A second proven advantage is the architecture boundary between interface, orchestration, and data surfaces. During development, the team was able to redesign auth, harden guest access, improve analytics, extend teaching-brief logic, and refine material processing without collapsing the entire system into one tangled codepath. The dedicated Python backend, the queue-backed material worker, and the Supabase policy and migration layer gave the product room to evolve. That separation reduced coupling and made later refinements more manageable, especially once the project moved beyond early vertical slices into more operational work.

The development record also shows a clear advantage in teacher-centered product thinking. Many student-facing educational demos become generic once they add AI, but this project repeatedly strengthened the instructor's side of the system instead: editable blueprint governance, assignment review flows, a teacher chat monitor, adaptive teaching briefs, class intelligence snapshots, and student-preview mode inside the same class shell. That matters because it gives the platform a practical classroom identity. The product is not merely helping students ask questions; it is also helping teachers prepare, supervise, interpret, and intervene.

Another advantage that became more convincing over time is demonstrability. Guest mode is not a static showcase or a scripted walkthrough. It is a real sandbox that clones seeded class data, supports role switching, and cleans up after itself. That is a strategic advantage for an academic project because it makes the system easier to evaluate honestly. A reviewer can interact with the same core flows that real users would see, while the system still preserves data isolation and resource control.

Finally, the project benefited from disciplined documentation and verification. The current repo contains aligned design, architecture, deployment, UI, and subsystem guides, plus layered web, backend, and end-to-end tests. The commit history shows that documentation was updated alongside major changes rather than being postponed until submission week. That habit reduced drift between what the system claimed to do and what it actually did, which is one of the clearest signs of engineering maturity in the repository.

7.3 Reasonable next-stage directions

The most practical next step for the platform is to connect insight more directly to follow-up teaching action. The current system can already identify class weaknesses, summarize misconceptions, and recommend next steps through the teaching brief and class intelligence dashboard. A sensible extension would be to let those recommendations lead into teacher-reviewable follow-up work such as targeted quizzes, intervention flashcard sets, or focused chat assignments tied to weaker topics. Because the platform already has blueprint grounding, analytics snapshots, assignment flows, and editable generation surfaces, this would extend the current design rather than introduce a new product direction.

Another reasonable direction is to broaden analytics from a single-class view into a more longitudinal teaching support view. At present, the system is already capable of summarizing one class coherently. A later iteration could help teachers compare patterns across classes, track recurring misconceptions over time, and relate assigned activities more clearly to later student improvement. This would remain consistent with the current teacher-support focus of the platform.

The third direction is operational rather than pedagogical: stronger visibility into system health. The platform now includes provider fallback, queue-backed workers, Edge Functions, sandbox quotas, background cleanup, and multi-surface deployment concerns. Over time, it would be useful to expose worker status, guest-capacity pressure, provider failures, and environment alignment checks more explicitly. That would not change the classroom workflow directly, but it would make the system easier to operate and maintain as the project continues to mature.

8 Conclusion

This project was not designed as a generic AI study tool with classroom branding added afterwards. It was designed around a more specific educational idea: teachers should be able to turn their own materials into a Course Blueprint, keep editorial control over what is published, and then use that blueprint to guide what students learn, what the AI can generate, and what the teacher can later review. That choice gave the platform a coherent center. Materials, activities, chat, review, teaching brief, and analytics all connect back to the same class-specific instructional object.

The current repository is also stronger than a one-path prototype because the implementation

supports that idea at system level. The platform combines a dedicated Python backend for AI orchestration, queue-backed material processing through Supabase, policy-enforced data access, teacher-facing insight snapshots, and a guest sandbox model that allows realistic evaluation without using real classroom data. These choices add complexity, but they also make the project more credible as a working software system.

The development process also showed that the quality of the final result depended heavily on hardening and revision. Guest mode needed redesign and quota control. Authentication and recovery needed repeated refinement. Analytics, chat, and material flows needed to be normalized so they behaved like stable product features rather than raw AI outputs. This was an important part of the project outcome: progress came not only from adding features, but from improving reliability, consistency, and maintainability.

Taken together, the platform represents a serious and well-scoped course group project. It has a clear educational purpose, a credible technical architecture, meaningful GenAI integration, and a development record that shows real attention to testing, documentation, and deployment practice. The current implementation also leaves room for careful future improvement without overstating what has already been achieved.

9 User and Operations Manual

9.1 Administrator Manual (Deployment and Operations)

The administrator role in this platform is centered on environment setup, Supabase project administration, release deployment, and operational data management. In practice, administrators use the deployment pipeline, Supabase Dashboard, SQL editor, environment configuration, and health checks to operate the system across the web app, Python backend, Edge Functions, and hosted Supabase project.

A. Initial platform setup

1. Create and configure the target Supabase project.
2. Enable email/password auth and email confirmation.
3. Enable Anonymous Auth if guest mode is required.
4. Set the Site URL and redirect URLs for local and hosted environments.
5. Apply the active database migrations in `supabase/migrations`.
6. Deploy the two required Edge Functions:
   - `material-worker`
   - `guest-sandbox-cleanup`
7. Configure required secrets for the backend and both Edge Functions.
8. Deploy the web app and Python backend.
9. Verify the live environment and smoke-test the deployed product.

B. Core operational responsibilities

- Maintain environment variables for the web app, backend, and Edge Functions.
- Apply the branded confirmation and recovery email templates in Supabase.
- Ensure `PYTHON_BACKEND_URL` and backend API-key alignment across services.
- Monitor guest-mode capacity and cleanup health.
- Run post-deploy smoke tests after releases.
- Investigate common failure modes such as:
  - guest entry failures;
  - materials stuck in `processing`;
  - auth links redirecting to the wrong domain;
  - web UI loading while AI features fail due to backend or provider issues.

C. Common operating commands

Local development and verification

```
pnpm install
pip install -r backend/requirements.txt
pnpm dev
uvicorn app.main:app --app-dir backend --host 0.0.0.0 \
  --port 8001 --reload
pnpm test
python3 -m unittest discover -s backend/tests -p 'test_*.py'
```

Supabase deployment and schema operations

```
npx supabase db push
npx supabase functions deploy material-worker
npx supabase functions deploy guest-sandbox-cleanup
```

Frontend deployment

```
cd web
npx --yes vercel --yes --prod
```

Backend health verification

```
curl https://<backend-domain>/healthz
```

D. Useful Supabase Dashboard and SQL operations

The Supabase Dashboard is the main administrative control surface for account support, data inspection, and operational maintenance. Authentication issues can be handled in **Authentication -> Users**, storage artifacts can be reviewed in **Storage**, and targeted data operations can be executed through the **SQL Editor**. The following queries are directly aligned with the current schema.

Inspect user accounts, confirmation state, and profile roles

```
select
  u.id,
  u.email,
  u.email_confirmed_at,
  p.display_name,
  p.account_type
from auth.users u
left join public.profiles p on p.id = u.id
order by u.created_at desc;
```

Register a platform administrator

```
insert into public.admin_users (user_id)
values ('<user-uuid>'::uuid)
on conflict (user_id) do nothing;
```

Update a user's display name

```
update public.profiles
set display_name = '<new-display-name>'
where id = '<user-uuid>'::uuid;
```

Review classes and join codes

```
select id, title, subject, level, join_code, archived, created_at
from public.classes
order by created_at desc;
```

Inspect material-processing state and latest job status

```
select
  m.id,
  c.title as class_title,
  m.title as material_title,
  m.status,
  j.stage,
  j.attempts,
  j.last_error,
  m.created_at
from public.materials m
join public.classes c on c.id = m.class_id
left join public.material_processing_jobs j on j.material_id = m.id
order by m.created_at desc;
```

Review guest sandboxes and manually mark a sandbox for cleanup

```
select id, user_id, class_id,
       status, guest_role, expires_at, last_seen_at
from public.guest_sandboxes
order by created_at desc;

update public.guest_sandboxes
set status = 'expired', expires_at = now()
where id = '<sandbox-uuid>'::uuid;
```

Run a manual cleanup pass from the SQL editor when required

```
select public.cleanup_expired_guest_sandboxes(50);
```

9.2 Instructor Manual

A. Create an account and sign in

Public auth can start either from the landing-page modal flow or from the dedicated fallback routes `/login`, `/register`, and `/forgot-password`. Confirmation and password-recovery emails return through the same server-side callback route before the user is redirected back into the appropriate auth surface.

1. Open the deployed application.
2. Choose **Create an account**.
3. Select the **teacher** account type.
4. Enter email and password.
5. Verify the email using the confirmation link.
6. Sign in and open the **Teacher Dashboard**.

B. Create and manage a class

1. From the Teacher Dashboard, click **Create class**.
2. Fill in class title and optional metadata such as subject and level.
3. Submit the form.
4. The system creates the class and generates a join code.
5. Share the join code with students so they can enroll from the student side.

C. Upload teaching materials

1. Open the target class.
2. Use the materials upload form to upload PDF, DOCX, or PPTX files.
3. Wait for the system to process the material.
4. Monitor status as `processing`, `ready`, or `failed`.
5. When the material is ready, use the actions menu to preview, download, or delete it.

D. Generate and publish the Course Blueprint

1. Open the class blueprint page.
2. Trigger blueprint generation after materials are ready.
3. Review the generated summary, topics, and objectives.
4. Edit the blueprint draft where needed.
5. Approve the draft and move it into overview.
6. Publish the blueprint when it is ready for student use.

E. Create learning activities

The current release supports three main activity types:

- chat assignments;
- quizzes;
- flashcards.

To create an activity:

1. Ensure a published blueprint exists.
2. Open the relevant activity creation route from the class page.
3. Generate or configure the activity.
4. Review and edit the content where applicable.
5. Publish or assign it to the class.

F. Review submissions

1. Open an assignment's review page.
2. View student recipients, completion states, and submitted content.
3. For chat assignments, inspect transcript and reflection content.
4. For quiz assignments, inspect scores and answers.
5. For flashcards, inspect review statistics and score summaries.
6. Save teacher feedback for the selected submission.

G. Monitor class progress and teaching support

1. Open the class overview to see the adaptive teaching brief.
2. Refresh the brief when more activity data becomes available.
3. Open **Insights** for the class intelligence dashboard.
4. Review charts, student summaries, AI narrative, and intervention suggestions.
5. Use the natural-language data query panel for custom chart-oriented questions.

H. Preview the student experience

1. Open the class page as a teacher.
2. Activate student preview mode using the student-view route state.
3. Inspect how published blueprint, AI chat, and assignment widgets appear to students.
4. Exit preview mode to return to the teacher control surface.

9.3 Student Manual

A. Create an account and sign in

Students use the same shared auth surface as teachers. They can start from the landing-page modal or from the dedicated auth pages, and email confirmation/recovery links are completed through the same server-side callback path before the final redirect back into the app.

1. Open the deployed application.
2. Choose **Create an account**.
3. Select the **student** account type.
4. Enter email and password.
5. Verify the email using the confirmation link.
6. Sign in and open the **Student Dashboard**.

B. Join a class

1. From the Student Dashboard or sidebar, open **Join class**.
2. Enter the join code provided by the instructor.
3. Submit the form.
4. After joining successfully, open the class workspace.

C. Use the student dashboard

The student dashboard organizes assignments into categories such as:

- due now;
- upcoming;
- completed.

Students can use the dashboard to quickly identify overdue or active work and jump directly into the relevant class or assignment.

D. Navigate the class workspace

Within a class, students can access:

- AI chat;
- chat assignments;
- quizzes;

- flashcards;
- the published blueprint.

The workspace is intentionally more focused and less administratively dense than the teacher view.

E. Use AI chat

1. Open the class page and select the AI chat widget or direct chat view.
2. Ask questions related to the class materials and published blueprint.
3. Review the grounded AI response.
4. Continue the conversation as needed.

AI chat becomes available once the instructor publishes the blueprint, ensuring that responses are grounded in approved class content.

F. Complete assignments

1. Open the assigned activity from the dashboard or class workspace.
2. For chat assignments, complete the conversation and submit the required reflection.
3. For quizzes, answer the questions and submit the attempt.
4. For flashcards, complete the review session and submit progress.
5. Revisit the assignment later to see updated status such as submitted or reviewed.

G. Use the published blueprint

1. Open the blueprint widget or blueprint page inside the class.
2. Review topics and learning objectives.
3. Use the blueprint as a study guide before attempting quizzes or asking AI chat questions.

H. Manage account settings

1. Open **Settings**.
2. Update display name if needed.
3. Use the two-step password change flow with verification code if required.
4. Open **Help** for role-specific FAQ and suggested next steps.

9.4 Supplementary Guest Demo Flow

In addition to the required admin, instructor, and student manuals, the guest workflow is worth documenting because it is a significant implemented feature.

1. From the landing page, choose **Continue as guest** when guest mode is enabled.
2. The system provisions an anonymous authenticated session and clones sandbox data.
3. Explore the classroom in teacher view or switch to student view.
4. Reset the sandbox if a fresh walkthrough is needed.
5. Create a permanent account from a clean state if you want to move into normal usage.

A Repository Evidence Used for This Report

This report was prepared against the current repository state using the following main sources:

- top-level documents:
  - `README.md`
  - `ARCHITECTURE.md`
  - `DESIGN.md`
  - `UIUX.md`
  - `DEPLOYMENT.md`
  - `AGENTS.md`
  - `CLAUDE.md`
- subsystem documentation:
  - `web/README.md`
  - `backend/README.md`
  - `supabase/README.md`
- course materials:
  - `project_overivew.pdf`
  - `project_background.pdf`
- implementation evidence:
  - key web routes and server actions
  - backend service modules
  - Supabase Edge Functions
  - migration history
  - git commit history and merge history
  - test suites and E2E specs

B Submission and Live System Links

- Live product: ai-stem-learning-platform-group-8.vercel.app
- Demo video: youtu.be/xsq_rRhuclY
- Primary development repository: HaochenFa/AI-Learning-Platform-for-STEM
- Organization submission repository: comp3122-2526sem2/groupproject-team_8